“Exit the MazeTrix” Unity Project Documentation

TABLE OF CONTENTS

1. Gameplay....3

1.1. General....3

1.1.1. BombCounter.cs....3

1.1.2. BombObject.cs....3

1.1.3. ExplosionDamage.cs....3

1.1.4. HealthBar.cs....3

1.1.5. HungerBar.cs....3

1.1.6. SpawnItem.cs (ItemSpawner)....3

1.1.7. TrapHiding.cs....4

1.2. Enemy....4

1.2.1. EnemyCombat.cs....4

1.2.2. EnemyPatrol.cs....4

1.2.3. EnemyScaling.cs....4

1.3. Player....5

1.3.1. PlayerController.cs....5

1.3.2. PlayerAttack.cs....5

1.3.3. PlayerPlaceBomb.cs....5

1.3.4. PlayerXP.cs....5

1.3.5. SpriteRotator.cs....5

2. Managers....6

2.1. AuthManager.cs....6

2.2. FirebaseAIManager.cs....6

2.3. MenuManager.cs....6

2.4. PlayerSessionData.cs....6

2.5. SceneLoader.cs....7

3. Map Generation....7

3.1. Chunks....7

3.1.1. ChunkDefinition.cs....7

3.1.2. ChunkRecord.cs....7

3.1.3. ChunkManager.cs....7

3.1.4. SpawnItemsInChunks.cs....7

3.2. Maze....8

3.2.1. DungeonGenerator_v4.cs....8

3.2.2. SpawnPlayerInMaze.cs....9

4. User Interface..... 9
4.1. MainMenuOnLoginSuccess.cs..... 9
4.2. OpenPauseMenu.cs..... 9
4.3. QuitGame.cs..... 10
4.4. RetrieveSurvivalTime.cs..... 10
4.5. ShowAIResponse.cs..... 10
4.6. SurvivalTime.cs..... 10
4.7. UpdateLevelDisplay.cs..... 10
4.8. XPBar.cs..... 10

1. Gameplay

1.1. General

1.1.1. BombCounter.cs

- Purpose: manage player bomb inventory and UI text.
- Inspector/public: `bombText`, `bombCount`.
- Notes: Persists bomb count to `PlayerSessionData` and `AuthManager.SaveBombCount`.

1.1.2. BombObject.cs

- Purpose: contains bomb life-cycle coroutine, waits (animation), rescans A* graph, then destroys itself.
- Notes: Calls `AstarPath.active.Scan()`. Does cause a stutter because of Scan().

1.1.3. ExplosionDamage.cs

- Purpose: apply damage / tile destruction upon explosion area trigger.
- Inspector/public: `wallMap` (walls Tilemap), `floorMap`, `floorTile` for replacement.
- Behavior: damages `Player` and `Enemy` tagged objects; attempts to destroy wall tiles and avoids creating exposed holes by checking neighbors. Places `floorTile` after destroying wall.
- Notes: Requires correct Tilemap references and floorTile asset.

1.1.4. HealthBar.cs

- Purpose: manage player health, death events and syncing to `PlayerSessionData`.
- Inspector/public: `healthSlider`, `maxHealth`, `currentHealth`, `OnDeath`.
- Notes: `TakeDamage`, `Heal` update slider and call `AuthManager.SavePlayerStats` via `PlayerSessionData` pattern.

1.1.5. HungerBar.cs

- Purpose: hunger tick-down over time, alerting and saving to Firebase when low/empty.
- Inspector/public: `hungerSlider`, `maxHunger`, `currentHunger`, `hungerTickSeconds`, `OnHungerEmpty`.
- Notes: Uses `PlayerSessionData`/`AuthManager.SavePlayerStats` to persist.

1.1.6. SpawnItem.cs (ItemSpawner)

- Purpose: scene item spawner (non-chunked), periodically spawns items at random positions within ranges.

- Inspector/public: `itemsToSpawn[]`, `spawnRangeX/Y`, `timeBetweenSpawns`.
- Notes: Checks collision via `Physics2D.OverlapCircle` before instantiating.

1.1.7. TrapHiding.cs

- Purpose: hides trap sprite by setting alpha to 0 until player triggers it, reveals on player collision.
- Notes: Assumes a `SpriteRenderer` on same GameObject, prefab for trap already configured with the expected components

1.2. Enemy

1.2.1. EnemyCombat.cs

- Purpose: enemy health, damage handling, death bookkeeping XP + AI log
- Important fields: `maxHealth`, `currentHealth`, `animator`.
- No dependencies
- Notes: Calls `FirebaseAIManager.Instance?.UpdatePlayerLog(...)` and `PlayerXP.Instance?.AddKillXP()` on death.

1.2.2. EnemyPatrol.cs

- Purpose: sight, chasing, return-to-home, attack behavior grid-bound checking.
- Important inspector fields: `returnDelay`, `sightRange`, `attackCooldown`, `attackRange`, `gridMover`.
- Dependencies: A* Pathfinding Project components `AIDestinationSetter`, `AIPath` and `ProceduralGridMover` for bounds checking.
- Notes: Pauses movement if outside grid bounds; creates a hidden `homeTarget` at startup which is equal to initial position.

1.2.3. EnemyScaling.cs

- Purpose: periodic scaling of enemy stats using `SurvivalTime`.
- Important fields: `updateInterval`, `healthScalePerMinute`, `speedScalePerMinute`, `maxMultiplier`.
- No dependencies
- Behavior: finds `SurvivalTime` in scene and adjusts `EnemyCombat` health, `EnemyPatrol` parameters and `AIPath.maxSpeed`.

1.3. Player

1.3.1. PlayerController.cs

- Purpose: player movement input + periodic save of player position & chunk data, also receives player position updates from chunk generator
- Inspector/public fields: `baseSpeed`, `buffedSpeed`, `savePositionInterval`.
- Important methods:
  - `OnPlacedByGenerator(Vector3)` : set position and apply `PlayerSessionData` stats (health/hunger).
  - `SaveCurrentPosition`/`SaveCurrentChunkData`: call `AuthManager` to save to database
- Notes: Automatically reads `PlayerSessionData.Instance` if present. Keeps `_placedByGenerator` to avoid overwriting generator placement.

1.3.2. PlayerAttack.cs

- Purpose: melee attack logic (trigger animation, knockback and damage).
- Inspector/public: `animator`, `weapon`, `transform`, `attackRange`, `enemyLayers`, `attackStat`, `attackCooldown`.
- Notes: Uses `Physics2D.OverlapCircleAll` and `EnemyCombat.TakeDamage`. Logs actions to `FirebaseAIManager`.

1.3.3. PlayerPlaceBomb.cs

- Purpose: spawn bomb prefab at `bombSpawnPoint` when player presses key.
- Inspector/public: `bombCounter`, `bombPrefab`, `bombSpawnPoint`.
- Notes: Checks for collisions before placing, decrements `BombCounter`.

1.3.4. PlayerXP.cs

- Purpose: spawn bomb prefab at `bombSpawnPoint` when player presses key.
- Inspector/public: `bombCounter`, `bombPrefab`, `bombSpawnPoint`.
- Notes: Checks for collisions before placing, decrements `BombCounter`.

1.3.5. SpriteRotator.cs

- Purpose: rotate player sprite to face movement direction using input axes, keeps movement axes the same while enabling the sprite

component to turn (i.e. up arrow will always correspond to up regardless of sprite direction)
- Notes: simple helper reading `Input.GetAxisRaw`.

2. Managers

2.1. AuthManager.cs

- Purpose: Firebase Authentication + Firestore profile load/save helper.
- Inspector fields: TMP fields for email/password UI and `statusText`.
- Singleton: `AuthManager.Instance`
- Key methods: `SignUp`, `Login`, `Logout`, `CreateUserProfile`, `LoadUserProfile`, `SavePlayerStats`, `SaveBombCount`, `SavePlayerPosition`, `SaveXP`, `SaveWorldSeed`, `SaveCurrentChunkPosition`.
- Notes: Requires `FirebaseBootstrap.Ready` and proper Firebase packages. Updates `PlayerSessionData.Instance` when profiles are loaded.

2.2. FirebaseAIManager.cs

- Purpose: Uses Firebase.ai package to generate narrator dialogue. Keeps a short `playerlog` to track player actions. Displays responses via `ShowAIResponse` every `updateInterval` using the `playerlog` for context, then clears the log for the next response.
- Inspector/public: `systemInstructions`, `playerlog`, `updateInterval`, `enablePeriodicUpdates`, `responseDisplay`.
- Singleton: `FirebaseAIManager.Instance`.
- Behavior: `Start()` greets, ability to `SendPromptAsync` and periodic updates that send `playerlog`. Keeps `playerlog` trimmed to ~500 chars.
- Notes: Requires configured Firebase AI backend with sufficient tokens and ShowAIResponse UI.

2.3. MenuManager.cs

- Purpose: canvas & menu activation helper.
- Inspector/public: arrays `cavases`, `menus`, `initialCanvas`, `initialMenu`.
- Notes: `OpenCanvasByName`, `OpenMenu` and additive menu logic are provided. One canvas can hold multiple menus. Example: the Portal canvas holds the Login and Registration menus.

2.4. PlayerSessionData.cs

- Purpose: in-memory session object persisted across scenes via `DontDestroyOnLoad`.
- Singleton: `PlayerSessionData.Instance`.

- Contains user profile and gameplay state fields (health, hunger, xp, worldSeed, currentChunkX/Y, positionX/Y/Z).
- Use: populated by `AuthManager` after login and used by player components.

2.5. `SceneLoader.cs`

- Purpose: simple wrapper to call `SceneManager.LoadScene` by name or index; intended for UI buttons.

3. Map Generation

3.1. Chunks

3.1.1. `ChunkDefinition.cs`

- Purpose: ScriptableObject with generator defaults (tiles, size, seed, reposition setting).
- Key methods: `ApplyToGenerator(DungeonGenerator_v4 generator, Vector3Int? overrideOrigin)`, `ToRecord()`, `ApplyRecordAndGenerate`.
- Notes: `ApplyToGenerator` ensures a `SpawnPlayerInMaze` component exists on the generator.

3.1.2. `ChunkRecord.cs`

- Purpose: serializable minimal version of a chunk: `chunkCoord`, `originCell`, `seed`, `useRandomSeed`.

3.1.3. `ChunkManager.cs`

- Purpose: dynamic instantiation/unload of chunk prefabs determined by player's chunk coordinate and `renderDistance`.
- Inspector/public: `mazeGeneratorPrefab`, `defaultChunkDefinition`, `chunkSize`, `renderDistance`, player.
- Behavior:
  - Keeps `records` dictionary of `ChunkRecord`.
  - `SpawnNewChunk` generates a deterministic chunk seed (uses `PlayerSessionData.worldSeed` when available) and sets `DungeonGenerator_v4` fields.
  - `SpawnChunkFromRecord` reconstructs chunk from `ChunkRecord`.
  - `UnloadChunk` clears the tiles within generator bounds on shared `Tilemaps`
- Notes: calls `SpawnPlayerInMaze.ResetSpawnFlag()` before initial spawn to ensure only one reposition occurs.

3.1.4. `SpawnItemsInChunks.cs`

- Purpose: procedural item population per chunk using chunk seed.

- Inspector/public: `itemsToSpawn[]`, `itemsPerChunk`, `minDistanceBetweenItems`, `spawnCheckRadius`, `obstacleLayers`, `chunkManager`, `floorTilemap`.
- Behavior:
  - Listens to `ChunkManager.records` and spawns items when chunk floor tiles are ready, uses `Random.InitState(record.seed ^ 12345)` to make placement repeatable.
  - Tracks `spawnedItemsByChunk` for cleanup on unload and `collectedItems` for persistence.
  - Adds `ItemCollectionTracker` to spawned objects; `OnDestroy` notifies spawner.
- Notes: items are assigned an ID composed of the chunk coordinates and an item number (chunkX_chunkY_itemNum) to flag each as picked up so that they don't regenerate when the same chunk is revisited.

3.2. Maze

3.2.1. DungeonGenerator_v4.cs

- Purpose: room+maze chunk generator based on "rooms and mazes" approach; paints into shared Tilemaps and keeps internal _tiles and _regions for validation and spawn queries.
- Inspector/public:
  - *Tilemaps*: `wallTilemap`, `floorTilemap`
  - *Tiles*: `wallTile`, `floorTile`, `closedDoorTile`, `openDoorTile`
  - *Dimensions & origin*: `width`, `height`, `originCell`, widths/heights must be odd for algorithm correctness
  - *Generation params*: `numRoomTries`, `roomExtraSize`, `windingPercent`, `extraConnectorChance`
  - *Chunk connectivity*: `enableChunkConnections`
  - *Seed controls*: `seed`, `useRandomSeed`
- Public API:
  - `Generate()` - generate based on current inspector/field settings (also invoked by `GenerateAsChunk` and `ChunkManager`).
  - `GenerateAsChunk(Vector3Int chunkOrigin)` - set origin then call `Generate`.
  - `SetSeed(int)` - deterministic RNG.
  - `TryGetFirstRoomSpawnCell(out Vector3Int)` - find center cell for first room; uses `GetNearestWalkableCell`.

- `IsFloorCell(Vector3Int worldCell)` - validate whether a given world cell is walkable according to `_tiles`.
- `GetNearestFloorCell(Vector3Int worldCell, int maxRadius)` - spiral search in internal `_tiles`.

- Notes:
  - When working with shared Tilemaps, `PaintToTilemaps` sets explicit tiles (or `null`) for each cell in the generator's chunk area to avoid wiping neighboring chunks.
  - If `enableChunkConnections` is true the generator carves a floor connection at the center of each edge then carves inward until connected. Default = true

3.2.2. SpawnPlayerInMaze.cs

- Purpose: handle player reposition/spawn validation after generator run, using generator internal map and optional A* nearest-node lookup.
- Inspector/public: `dungeonGenerator` (auto-assigned), `repositionPlayerAtFirstRoom`, `maxSearchRadius`.
- Behavior:
  - `TrySpawnOrRepositionPlayer()` schedules repositioning one frame later (to allow tilemaps to refresh).
  - Finds either saved position from `PlayerSessionData` or the first room center and validates with `DungeonGenerator_v4.GetNearestFloorCell`.
  - Finalizes reposition: sets `Transform`, resets Rigidbody2D velocities, calls player `OnPlacedByGenerator`, registers player with `ChunkManager`, rescans A* grid (`AstarPath.active.Scan()`), and opens main overlay via MenuManager.OpenCanvasByName.
  - Uses static flag `_playerRepositionedBySpawner` to avoid multiple repositions when many spawners run.
- Notes: Requires `Player` GameObject tagged "Player" to exist and `AstarPath` to be present for A* nearest-node fallback.

4. User Interface

4.1. MainMenuOnLoginSuccess.cs

- Purpose: hide/show a main menu transition button, starts disabled and exposed method `ShowMenuTransitionButton()`.

4.2. OpenPauseMenu.cs

- Purpose: toggle pause menu with ESC key and control `Time.timeScale`.
- Inspector/public: `pauseMenuUI`.
- Methods: `ResumeGame`, `PauseGame`, `PauseGameTime`, `ResumeGameTime`.

4.3. QuitGame.cs

- Purpose: quit application or exit Play mode if in editor. `ExitAndResetPlayerData()` resets `PlayerSessionData` then exits, called on the Quit button

4.4. RetrieveSurvivalTime.cs

- Purpose: UI script that reads `SurvivalTime.survivalTime` and formats it (hh:mm:ss) into the attached `TextMeshProUGUI`.

4.5. ShowAIResponse.cs

- Purpose: typewriter-style sentence-level display for AI narrator text.
- Inspector/public: `charactersPerSecond`, `delayBetweenSentences`, `initialDelay`.
- Methods: `SetAndDisplayResponse(string)` and `SkipToEnd()`.

4.6. SurvivalTime.cs

- Purpose: timer that counts seconds of survival, updates UI `TextMeshProUGUI`, persists last time via `PlayerSessionData` and `AuthManager`.
- Public API: `GetCurrentSurvivalTime()`, `SaveCurrentSurvivalTime()`, `StopTimer()`.

4.7. UpdateLevelDisplay.cs

- Purpose: fetches `PlayerSessionData.Instance.currentLevel` and shows it in UI, `UpdateCurrentLevel()` increments displayed level.

4.8. XPBar.cs

- Purpose: update a UI `Slider` to reflect `PlayerXP.Instance.CurrentXP / XPToNextLevel`.
- Notes: calls `UpdateBar()` on `Start` to match saved XP amount in `PlayerSessionData`